electronics

IMPACT FACTOR 2.6

CITESCORE 6.1

Article

A Rule-Based Method for Enhancing Burst Tolerance in Stateful Microservices

Kęstutis Pakrijauskas and Dalius Mažeika

Special Issue

New Advances in Cloud Computing and Its Latest Applications

Edited by

Dr. Pablo Alejandro Quezada-Sarmiento

MDPI

https://doi.org/10.3390/electronics14142752

electronics

MDPI

Article

A Rule-Based Method for Enhancing Burst Tolerance in Stateful Microservices

Kęstutis Pakrijauskas * and Dalius Mažeika

Department of Information Systems, Faculty of Fundamental Sciences, Vilnius Gediminas Technical University, Sauletekio Avn., 11, 10223 Vilnius, Lithuania; dalius.mazeika@vilniustech.lt

* Correspondence: kestutis.pakrijauskas@vilniustech.lt

Abstract

Microservice architecture enables the development of flexible, loosely coupled applications that support elasticity and adaptability. As data availability is critical in any application, maintaining consistent access becomes a top priority. However, this introduces complexity, particularly for stateful microservices, which are slower to adapt to sudden changes and can lead to data availability degradation. Resource overprovisioning may prepare systems for peak loads; however, it is an inefficient method of problem-solving. Similarly, dynamic scaling based on machine learning can underperform due to insufficient training data or inaccurate prediction methods. This paper proposes a rule-based method that combines write-scaling and load balancing to distribute burst workloads across multiple stateful microservice nodes while also vertically scaling a single node to meet rising demand. This approach reduces failure rates and extends operational time under burst conditions, effectively preserving the application's availability by providing more time for scaling. An experiment was performed to validate the proposed method of burst tolerance by workload distribution. The results showed that the proposed method enables a stateful microservice that sustains the burst load for nearly twice as long while further reducing the rate of failure.

Keywords: stateful microservice; availability; database; cluster; Kubernetes; failover; burst; write-scaling

check for updates

Academic Editors: George Angelos Papadopoulos and Pablo Alejandro Quezada-Sarmiento

Received: 5 June 2025
Revised: 1 July 2025
Accepted: 4 July 2025
Published: 8 July 2025

Citation: Pakrijauskas, K.; Mažeika, D. A Rule-Based Method for Enhancing Burst Tolerance in Stateful Microservices. *Electronics* **2025**, *14*, 2752. https://doi.org/10.3390/electronics14142752

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Microservices, a form of Service-Oriented Architecture, are specialized, loosely coupled, and autonomous services that make up an application. However, with low coupling and independence from other services comes an increased pace of code deployment and scalability, and there are inherited challenges such as the Federated Multidatabase [1,2] and Backup Availability Consistency [3]. The components of microservices are designed to manage failures rather than prevent them [4,5]. Due to their design with independence and scalability in mind, individual components of a microservice-based application can scale independently to match incoming workloads [4]. However, allocating excessive computing resources can lead to unnecessary costs, while insufficient provisioning may negatively impact the quality of service. The highly dynamic nature of workloads further complicates the challenge of efficient resource allocation. A sudden temporary workload increase, commonly referred to as a burst, might impact the quality of services provided and reduce availability metrics if not handled effectively. There is a challenge related to scheduling stateful containers, which further hinders the response to burst workloads. The

scheduling or rescheduling of stateful workloads might require data to be moved, which takes additional time before a container is healthy and ready to accept connections [6].

Given the dynamic nature of modern applications, a dilemma exists as to whether computing resources should be overprovided to meet occasional workload bursts or conserved, providing only enough to accommodate average workloads [7–9]. While one approach reduces the number of Serve Level Objective (SLO) breaches, the other conserves computing resources and reduces both costs and carbon footprint. A dynamic, data-driven approach would involve using machine learning (ML) to forecast workloads and scale microservices accordingly [10,11]. However, ML-based algorithms require both sufficient data and training time to develop models capable of predicting future resource demands. In the case of highly dynamic workloads, these predictions may lack the necessary precision, leading to suboptimal scaling decisions.

Furthermore, relational database management systems (RDBMSs) introduce additional challenges when handling burst workloads, as they involve managing the state, that is, through persistent data. Scaling such systems typically requires reading data from a disk, and in all scenarios involving highly available stateful microservices, data replication across nodes becomes essential [12,13]. Secondly, if a burst workload cannot be properly handled, database nodes that exceed their allocated memory are terminated. This leads to the loss of client connections to the database cluster and, in most cases, the termination of in-progress queries, resulting in a breach of service level objectives (SLOs). Additionally, the failure triggers a re-election process to designate a new primary node among the remaining database nodes, potentially causing further disruptions to availability.

ML-based workload prediction methods can enable the more accurate, proactive scaling of stateful microservices in anticipation of workload spikes and drops. However, if scaling is triggered too late or if insufficient resources are allocated, the microservice may underperform, potentially leading to a breach of availability service-level objectives (SLOs).

In this research, we propose a rule-based method to enhance resilience against burst workloads. Specifically, we evaluated whether the write-scaling and request-routing capabilities of a load balancer can be adapted to improve the burst tolerance of stateful microservices while one of the nodes is scaled vertically to meet the increased demand. Additionally, our previous research has demonstrated that it is possible to perform maintenance, including the vertical scaling, of Multi-Primary database clusters in an orchestrated container environment [14]. The results demonstrate that a database cluster can successfully handle sudden and significantly increased workloads, both in terms of sustained processing time and reduced error rates. The added capacity—particularly the extended time under load— enables more time to perform vertical scaling to process burst workloads, consequently allowing the avoidance of unnecessary dips into availability budgets.

This paper is organized as follows: Section 2 presents a literature review on the topics of stateful microservice availability and reliability and the principles that the proposed method is based upon; Section 3 describes the method we propose; Section 4 outlines the investigated architecture; Section 5 describes the experimental setup; Section 6 contains results of the experiment; and conclusions are presented in Section 7.

2. Background and Related Work

Traditionally, a sudden and significant increase in requests for a service is known as a burst. A burst is usually caused by a sudden increase in user activity due to unpredicted events. An example in Figure 1 illustrates the increased demand—a burst—on Michael Jackson's Wikipedia entry, both days in advance and after his death [15]. It is worth mentioning that a burst may be caused by a Denial of Service attack as well [15]. Such workload may lead to a decreased quality of service and violation of SLOs, including a loss

of availability. A burst can be unpredictable in terms of load volume and as an event in time [15]. The two primary properties of a burst are intensity and duration [15,16]. Intensity defines the number of requests, while duration defines the length of time that a service experiences the additional workload.

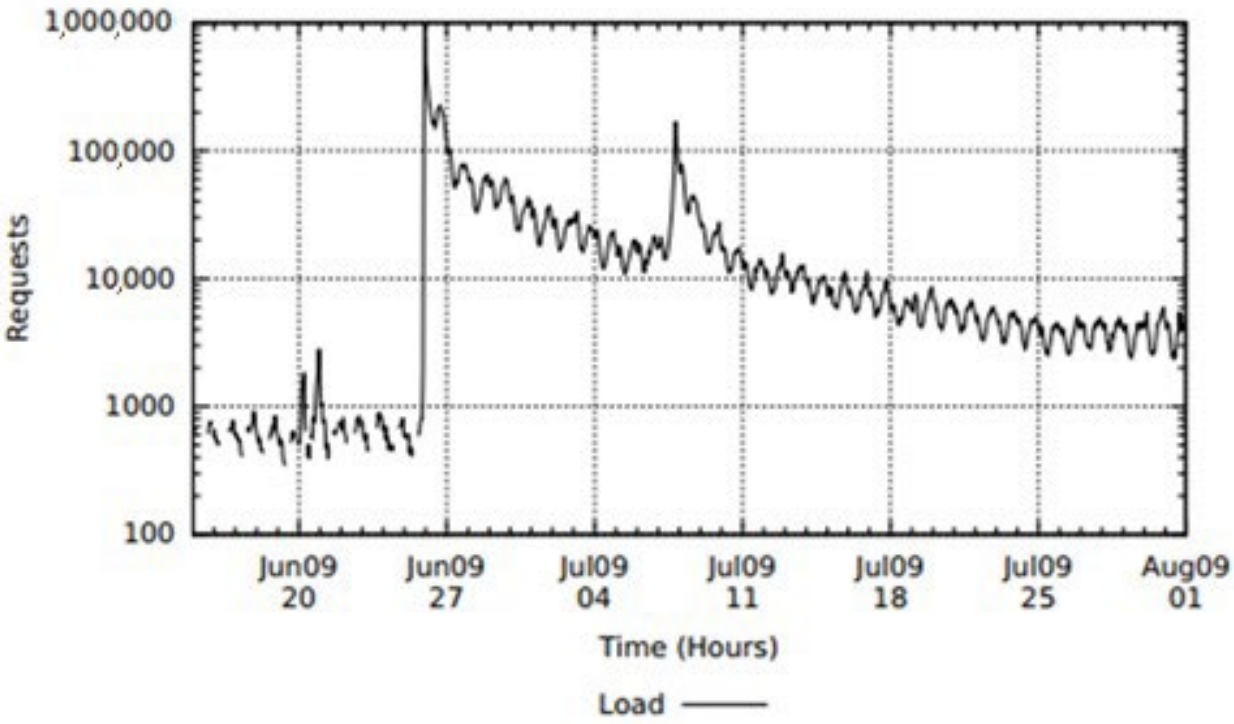


Figure 1. An example of requests. When a stateful service, not necessarily a microservice, is involved, a burst can have an additional property of data spikes, including an increased demand for a particular set of data. For example, in a scenario where the data are distributed across a partitioned database, a significant increase in demand for a particular subset of data can be considered a burst, while on the access layer, the number of overall requests remains the same [16].

There are different ways to measure burst workloads, for example, by calculating the standard deviation from the moving average, normalized entropy, or sample entropy [15]. For our experiment, we aimed to generate a linear synthetic burst. Thus, as Woodruf et al. suggested, we defined a burst as a sequence of x requests arriving within y time of each other [17].

An application or a service may scale up or undertake other means to process the load if bursts are predicted or periodic. However, workload bursts become a challenge if unpredictable: a system might not be able to operate properly under stress. Generally, two approaches can be taken to manage burst workloads: static or dynamic.

The static approach is quite simple: allocate a sufficient number of computing resources to handle a potential burst workload. This approach is more expensive; however, allocating enough computing resources to handle burst workloads may lead to the overprovision of resources. Resources for services can be optimally allocated to microservices without impacting the latency of user requests [18]. Baarzi and Kesidis were able to improve resource allocation by 22% while improving request latency by 20%. However, it is the unpredictability of burst workloads that poses a risk regardless of how optimal the resource allocation is.

The dynamic approach is data-driven: a burst is predicted, and a system scales up accordingly to handle the load. The second approach requires an elastic system and different mechanisms to predict upcoming bursts [11,19,20]. Lassnig et al. present an averages-based

model that provides an estimated probability that a burst is going to happen. However, as the authors say, the model needs optimization [11]. Iqbal et al. used the expectation-maximization method and Scale-Weighted Kmeans to partition service request logs in order to capture workload characteristics. The distribution of the characteristics was then used to compute the probability vector describing the distribution of incoming requests. It was then used to proactively auto-scale applications [19]. PBScaler, as proposed by Xie et al., collects the real-time performance metrics of applications and builds a correlation graph between microservices. The proposed scaling method achieves precision up to 0.92 using the Random Forest algorithm, and TopoRank, a random walk algorithm, was able to identify the likely bottlenecks [20].

A burst may exhaust various system resources, such as the Central Processing Unit (CPU), memory, network bandwidth, disk throughput, etc. While a lack of sufficient computing power, slow network, and low disk read speeds have an impact on the SLO, it is memory exhaustion that has the most significant impact on stateful microservice availability. A container is terminated when it reaches the limit of allocated memory [21]. Relational database management systems, for example, MySQL, use memory to cache data, query execution plans, monitor data, store client connection buffers and thread stacks, etc. [22]. Thus, during a burst, memory is exhausted both by the additional client connections and the increased demand to access cache data.

All nodes of Multi-Primary database management systems are equal, and all can process read and write requests at the same time. However, write scaling, which refers to the process of increasing the capacity or capability of a system to handle a larger volume of write operations, is not recommended: the improvement in write throughput, even with only two writer nodes, is usually limited [23]. The write on a database node is processed, and only the state update is transferred to the other nodes, so there is no need to reprocess the write again [24]. If one node is designated for processing write requests, only its memory is used to cache processing-related data, such as execution plans.

3. The Method of Burst Tolerance

In this section, we present the proposed method used to enhance the burst tolerance of stateful microservices. In orchestrated environments, containers are assigned limited computing resources, making them susceptible to exhaustion under high loads. This risk is further amplified by the burst-type workload. Stateful microservices require memory to process incoming requests, which are utilized for various purposes such as caching, connection buffering, and query execution. Consequently, there is a limit to the number of concurrent requests and clients a single node can handle. In clustered stateful systems such as databases, multiple nodes are available to share the workload.

We argue that write scaling may have a benefit in increasing the overall capacity for both read and write requests. Requests are processed only on one node; only the new state of data is transferred to the Secondary nodes. The memory utilization on the Secondary nodes is lower. Thus, there is room to process queries as the cache has space to keep query plans and temporary data. On the other hand, there is a risk of reduced throughput and replication conflicts as database nodes are likely to perform at the edge of capacity. However, if a stateful microservice is not capable of handling a burst workload, the choice has to be made as to which SLOs are to be breached: either accept the loss of throughput while minimizing or negating the loss of availability altogether or only accept the loss of availability but at a higher scale.

The proposed method aims to distribute the burst workload while stateful microservice nodes are scaled up vertically to match the increased demand for resources (Figure 2).

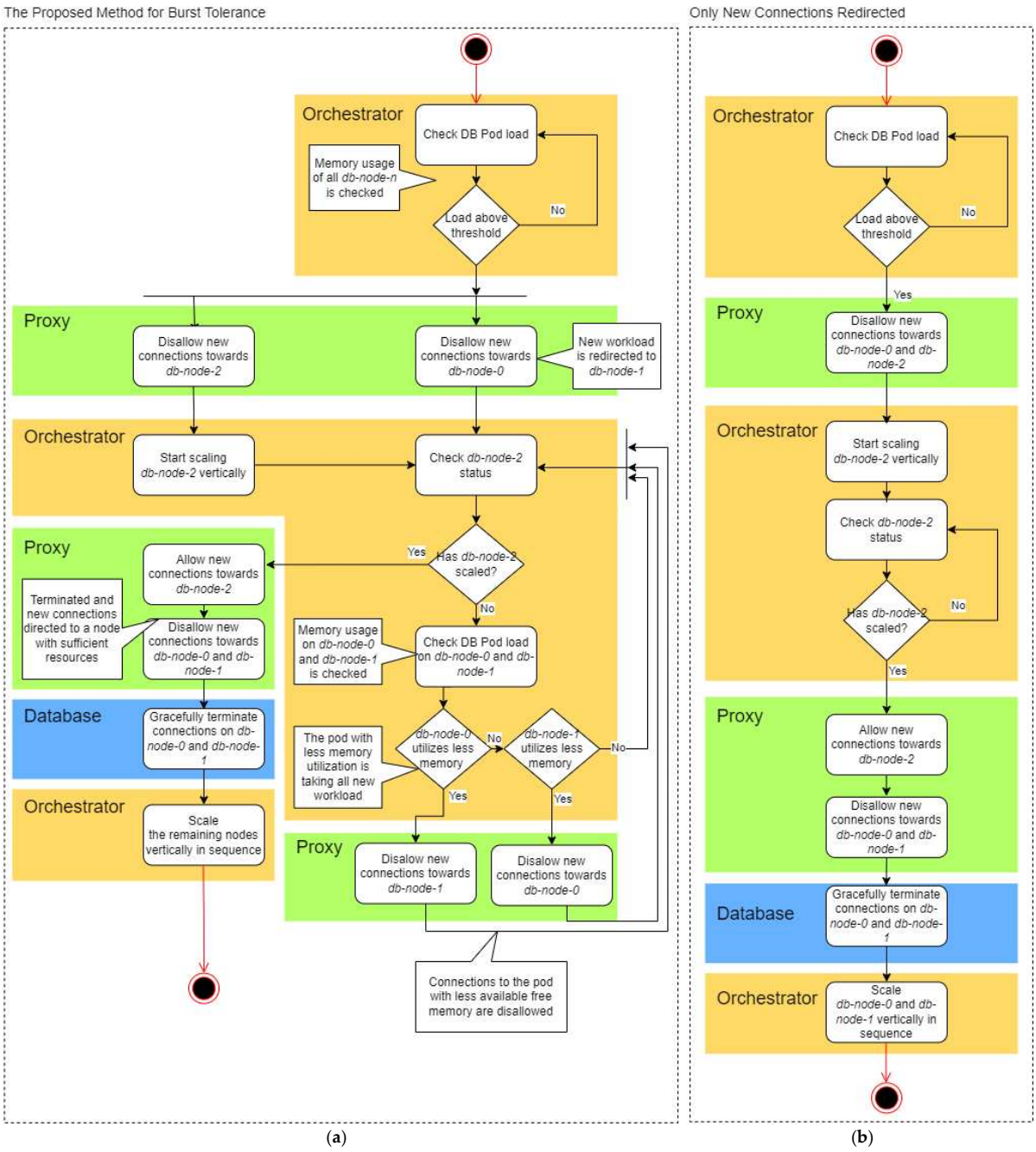


Figure 2. Flow diagram of proposed burst tolerance method (**a**) and alternative approach (**b**).

A stateful microservice cluster with a minimum of three nodes is required for the proposed method to function. We considered a cluster with three nodes *db-nodes-*$[0-n]$, where $n \geq 2$. The flow of the proposed rule-based method is shown in Figure 2a. The orchestrator, in the case of the Kubernetes operator, monitors resource usage on the stateful microservice nodes. Once the memory on the Pseudo-Primary node, *db-node-0*, is above the threshold, new connections are made for the *db-node-m* node, where $m \geq 2$, and *db-node-0* is disallowed by the database proxy. The *db-node-m* node, where $m \geq 2$, is prepared for increased demand, while an additional load is directed to the remaining node, *db-node-1.* The orchestrator starts the vertical scaling of the *db-node-m*, where $m \geq 2$. Then, the balancing between the remaining two or more nodes, the Pseudo-Primary *db-node-0* and *db-node-1*, begins: the orchestrator monitors memory usage on these two nodes and instructs the database proxy to disallow new connections to the node with higher memory utilization,

effectively balancing the load between two or more nodes. New connections are established with nodes that have more available memory. As memory utilization on the database nodes *db-node-0* and *db-node-1* changes, so does the node where new connections are directed. Already established connections are left intact. Once *db-node-m* is used, where $m \geq 2$, has been scaled, the orchestrator instructs the database proxy to allow new connections to be established towards that node and existing ones are gracefully moved from *db-node-0* and *db-node-1*: client connections are terminated by the database engine and reestablished towards node *db-node-m*, where $m \geq 2$ The graceful transfer of connections to the node that just finished scaling vertically is performed with negligible impact on availability. The remaining nodes are then scaled vertically in sequence by the orchestrator.

The threshold values and the new memory limit for the cluster can be selected in two ways. First, the values can be based on expert knowledge and would likely vary based on the individual or the team. Depending on the cluster size, the threshold could be 85 to 95 percent, while the memory limit could be increased by an additional 10 to 25 percent. ML-based prediction values could further optimize resource usage as the threshold and new memory limits could be set dynamically based on historical usage patterns.

Although there are more metrics that could be used to measure the utilization of a stateful microservice, the decision of whether to balance the load and initiate vertical scaling is based on memory usage. Other metrics, such as CPU utilization percentage, the number of requests, response latency, etc., could be used as well. However, considering the effects of container memory exhaustion, the degradation of the aforementioned metrics is not as impactful on availability SLO.

As an alternative, only new connections can be directed to a Secondary node. If only one node is designated as the Pseudo-Primary, the other nodes only handle the workload related to state transfer. Thus, their memory consumption is lower than that of a Pseudo-Primary node. There is no need to cache query execution plans, temporary data, etc. Therefore, a simpler approach would be to transfer only the new connections to the secondary node (Figure 2b). However, we do not expect this method to be on par with workload balancing. When only new connections are redirected, the remaining free memory on the original Pseudo-Primary node will not be utilized.

4. Implementation of the Method

The proposed method allowed us to temporarily distribute the load across database cluster nodes in the event of a workload burst. As shown in our previous work, workloads can be transferred between database nodes with negligible impact on availability [8]. As shown in Figure 3, the foundational components necessary for a stateful burst protector to work are the following:

1. A database cluster setup for Multi-Primary replication;
2. A proxy layer that can direct client requests to a specific node;
3. A connection pool to enable low-latency database connections.

The components necessary for transparent failover of client sessions between database nodes are as follows:

4. Client connection termination mechanism connections (a stored procedure);
5. A retry mechanism on the client's side to handle termination.

The components necessary for a stateful burst protector are as follows:

6. A burst-protector mechanism for triggering a failover and node scale-up in the event of a burst;
7. A modified failover orchestrator to oversee the scale-up and transfer workload to an appropriately sized node with negligible impact on availability.

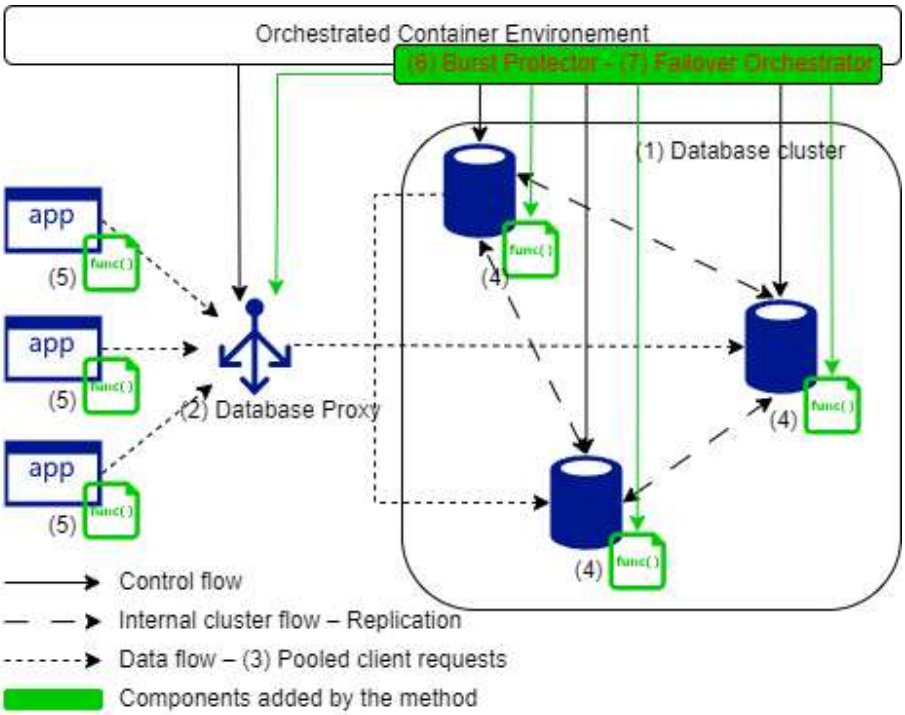


Figure 3. Components that are required to implement the proposed method.

Using Multi-Primary replication, a distributed database cluster maintains uninterrupted service for clients during an unplanned failover of connections between database'nodes.

The proxy layer with the capability to inspect the coming queries enables the balancing of requests between nodes while the burst workload is being handled. In addition, it will be used to transfer requests to a scaled-up node with a near-zero loss of availability. This was taken from our previous work and is shown in Algorithm 1.

Algorithm 1: Direct Database Connections

Input: *dbNode, connectionAction*
Output: *returnMsg*

```
1: procedure directDBConnections
2:     /* higher weight increases the node priority */
3:     defaultNodePriority ← 10
4:     if connectionAction = 'disallow' then
5:         // weight valued 0 disallows new connections towards the node
6:         nodePriority ← 0
7:         setNodePriority(dbNode, nodePriority)
8:         returnMsg ← 'connectionsDisallowed'
9:     end if
10:    if connectionAction = 'allow' then
11:        setNodePriority (dbNode, defaultNodePriority)
12:        returnMsg ← 'connectionsAllowed'
13:    end if
14:    return returnMsg
```

The failover orchestrator, introduced in our previous work, must be modified to make sure that the nodes are scaled up in a certain order. It is updated to perform a failover and drain connections on a single node. Its algorithm is depicted in Algorithm 2.

Algorithm 2: Prepare Node for Scaling

```
    Input: dbNode, proxyNode
    Output: exitStatus
1: procedure prepareNodeForScaling
2:     clientConnectionState ← proxyNode.directDBConnections(dbNode, 'disallow')
3:     if clientConnectionState = 'connectionsDisallowed' then
4:         terminateClientConnections.state ← dbNode.terminateClientConnections()
5:     end if
6:     if terminateClientConnections = 'connectionsDrained'
7:         return SUCCESS
8:     end if
```

Given that there are at least three nodes in the cluster, one node is scaled up, while the workload is balanced between the other two nodes. As shown in Algorithm 3, the node to direct the burst workload is selected based on the memory utilization of the nodes. The workload is directed to the node with lower memory utilization. The balancing of the workload between the two nodes is performed while the third node is being scaled up. Once the third node is scaled and ready to accept connections, the balancing process returns a message indicating success, and the burst protector procedure continues managing the database cluster and workload.

Algorithm 3: Balance Workload

```
    Input: dbNodeList, scaledDBNode, proxyNode, memoryThreshold
    Output: exitStatus
1: procedure workloadBalancer
2:     /* call a generic procedure to get initial scaled node status */
3:     scaledNodeStatus ← getNodeStatus(scaledDBNode)
4:     /* balance the connections while the node scaling is not complete */
5:     while scaledNodeStatus ≠ 'scalingComplete'
6:         for each: dbNode ∈dbNodeList
7:             /* call a generic procedure to get memory usage and add it to a list */
8:             dbNodesMemUsage[] ← getNodeMemUsage(dbNode)
9:         end for
10:        /* find the node with the highest memory utilization by iterating the list */
11:        dbNodeMemHigh ← dbNodesMemUsage [0]
12:        for each: dbNodeMemUsage ∈ dbNodesMemUsage[]
13:            if dbNodeMemHigh.memUtilization < dbNodeMemUsage.memUtilization then
14:                dbNodeMemHigh ← dbNodesMemUsage
15:            end if
16:        end for
17:        nodeConnectivityState ← proxyNode.directDBConnections(dbNodeMemHigh,nodeName 'disallow')
18:        if nodeConnectivityState = 'connectionsDisallowed' then
19:            /* call a generic procedure to check scaled node status */
20:            scaledNodeStatus ← getNodeStatus(scaledDBNode)
21:        end if
22:    end while
23:    return SUCCESS
```

The burst protection mechanism, shown in Algorithm 4, tracks memory usage and triggers both the balancing of client sessions and scale-up operation when a certain threshold

of memory utilization is reached on the Pseudo-Primary database node. Once the threshold is reached, the burst protector initiates the vertical scaling operation of the Secondary node with the highest memory utilization and client connection balancing between the Pseudo-Primary database node and the remaining node. The database cluster, or rather the two balanced nodes, has more memory with which to handle the burst if the Secondary node with the highest memory utilization is scaled first. When the scaling up is finished, all client connections are transferred to the node with more computing resources from the remaining two nodes. These two nodes are then scaled up to match the increased demand.

Algorithm 4: Burst Protection Mechanism

Input: *dbNodeList, pseudoPrimaryNode, memoryThreshold*
Output: *exitStatus*

```
1: procedure burstProtector
2:     /* call a generic procedure to get memory usage on pseudo-primary node */
3:     dbNodeMemUsage← getNodeMemUsage(dbNode)
4:     /* loop is exited once memory utilization on pseudo-primary node is above the threshold */
5:     while dbNodeMemUsage ≤ memoryThreshold
6:         /* only check memory utilization on pseudo-primary node */
7:         /* while memory utilization is below the threshold */
8:         dbNodeMemUsage← getNodeMemUsage(pseudoPrimaryNode)
9:     end while
10:    /* collect memory usage on the nodes */
11:    for each: dbNode ∈ dbNodeList
12:        if dbNode.name ≠ pseudoPrimaryNode.name then
13:            /* call a generic procedure to get memory usage */
14:            dbNodesMemUsage[] ← getNodeMemUsage(dbNode)
15:        end if
16:    end for
17:    /* find the node with the highest memory utilization by iterating the list */
18:    dbNodeMemHigh ← dbNodesMemUsage[0]
19:    for each: dbNodeMemUsage ∈ dbNodesMemUsage[]
20:        if dbNodeMemHigh.memUtilization < dbNodeMemUsage.memUtilization then
21:            dbNodeMemHigh ← dbNodesMemUsage
22:        end if
23:    end for
24:    /* call a generic procedure to scale the node up */
25:    call performDBNodeMaintenance(dbNodeMemHigh)
26:    /* start balancing sessions between two remaining nodes */
27:    balancedNodesList ← dbNodeList.removeItem(dbNodeMemHigh)
28:    call workloadBalancer(balancedNodesList, dbNodeMemHigh,proxyNode, memoryThreshold)
29:    return SUCCESS
```

There are a couple of advantages to the proposed method compared to other techniques to handle a burst:

- No reliance on historical data. Undoubtedly, workload prediction using ML is a powerful way to prepare for a burst. However, the prediction may be imprecise if there is not enough data to predict a burst. The proposed method gives an additional layer of protection against incidents related to stateful microservices that do not handle a burst properly. Expert knowledge or advice is sufficient to set the memory usage threshold.

- The method is predictable. A rule-based approach is transparent as the outcome is predefined by an understandable set of rules. In addition, such an approach does not require the additional computing resources that an ML-based approach would need.
- Appropriate resource allocation. Stateful microservices have sufficient resources to handle the baseline load. Additional resources are allocated only when there is a need to handle more load.

If no historical data are available, then the "traditional" approach is used to accept the loss of availability while a stateful microservice is scaled up. The proposed method allows, to a certain extent, the loss of availability to be minimized as a stateful microservice, even if it runs out of memory, as it can operate under increased loads for longer durations of time.

The other approach is throttling the clients [25]. However, the method we propose does not require any action to be taken on the client's side of a stateful microservice under burst conditions. This, in turn, follows the microservice paradigm of reduced coupling.

Of course, if a burst workload cannot be balanced between the database nodes due to resource constraints, there are not many choices but to accept the loss of availability.

A prototype of the proposed method is available on GitHub [26].

5. Evaluation of the Method

An experiment was conducted to evaluate the effectiveness of the proposed method under burst conditions, and this was compared against the out-of-the-box functionality of a MySQL Galera cluster in handling burst workload balancing. A prototype environment was built, incorporating the following necessary components: a database proxy, a database cluster, a burst protection orchestrator, and a client. A synthetically generated dataset, along with a synthetic workload, was used to simulate burst scenarios and assess system behavior. In our experiment, we focused on the following points:

- Operational time under burst—this is the time in which the nodes, other than the one being scaled up, are able to handle a burst workload. In other words, the time memory utilization increases from the set balancing threshold to the limit.
- The number of failed requests if scaling is successful. We presume that if one of the nodes does not scale on time, the loss of availability is too great. In addition, the way requests are handled during a burst operation is important. As suggested by Hauer et al., the percentage ratio of failed and successful requests will be the measure of availability [27].

The investigated architecture of the cluster is shown in Figure 4. The prototype environment, based on the investigated architecture, was setup on a three-node Kubernetes cluster in the Google Cloud Platform (GCP). The three nodes were identical: E2 cost-optimized instances with 6 GB of memory, 4 CPUs, and 50 GB of standard persistent disk (PD). The MySQL Galera cluster was chosen as this database management system has the capability of multi-master replication. ProxySQL was used as the database proxy.

The database client was setup on a virtual machine (VM) in GCP. The VM was an E2 cost-optimized instance with 4 CPUs and 8 GB of RAM, running Ubuntu 22.04. The failover orchestrator was setup on an E2 cost-optimized instance with 1 CPU and 1 GB of RAM, running Ubuntu 22.04. Software versions and resource allocation for the MySQL Galera cluster and ProxySQL nodes are listed in Table 1.

Table 1. Software versions and resource allocation.

Software and Version	Number of Pods	CPU Limit	Memory Limit
MySQL Galera Cluster 8.0.36	3	500 m	1024 MB
ProxySQL 2.2.5	2	400 m	256 MB

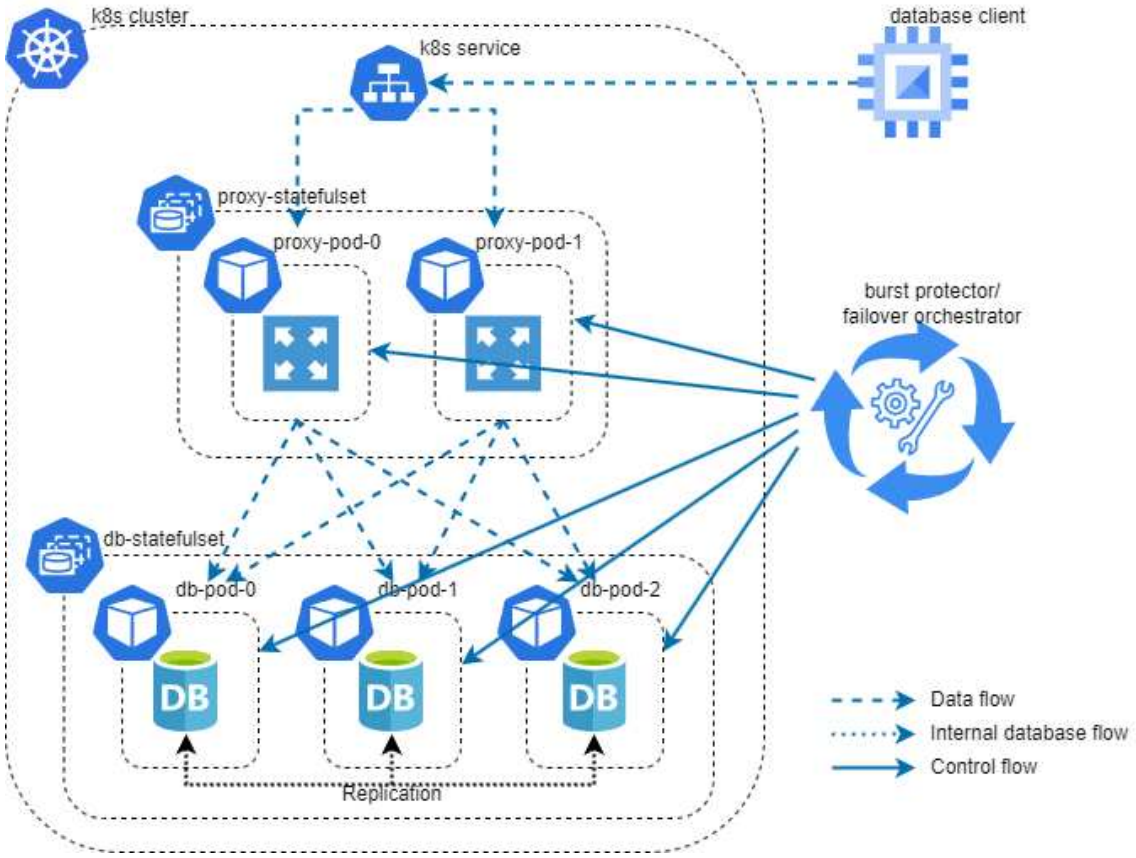


Figure 4. The architecture of the multi-master database cluster investigated.

The Yahoo! Cloud Serving Benchmark (YCSB) was used to generate both the synthetic data and the workload [28]. A table of 100,000 records was loaded into the database cluster. To keep the experiment as consistent as possible, we used the same data across the experiment runs. The workload generated was read-focused, with 83% of selects, 15% of updates, and 2% of inserts. Request distribution was set to uniform. Although the functionality of YCSB allowed requests to be generated in more realistic patterns and different workload ratios, the uniform request distribution and the read-focused workload were sufficient enough for validation if requests can be balanced across database nodes. The database driver on the client side was set to make three attempts to reconnect every 2 s.

Although data spikes are a property of a burst towards a stateful microservice, we did not consider it in our experiment. Since the database cluster used in the experiment was relatively small and the data were fully replicated across the nodes, the increase in requests only for a subset of data had a negligible impact on the overall results.

The base workload generated by YCSB included 10,000 operations and 160 threads with a throughput target of five operations per second. Insert queries were included in the base workload. It was intended to have this workload running for at least 120 s. The parameters for the base workload were selected to reach the memory utilization of the Pseudo-Primary database to 600 MB. However, in practice, due to limited control over how MySQL utilizes the memory, the base workload may peak between 580 MB and 620 MB.

As mentioned earlier, the duration and intensity are the main features of a burst. Since the duration property is expected to be mitigated by scaling and session balancing, we will evaluate the proposed method under different intensities.

The burst workload started 10 s after the base workload. A new client of five threads was added every 0.25 or 0.5 s, with target operations per second of 15, aimed at completing 2500 operations. Each client was intended to run for around 33 s. In total, 30 clients with

150 burst threads were started. With a new client added every 0.25 s, the peak of 150 threads is reached in, considering the overhead of code execution, approximately 7.5 s. With a new client added every 0.5 s, the peak is reached at roughly 15 s. As for the total number of requests, 2500 operations for over 30 client threads will amount to around 75,000 requests. The intensity of the burst is illustrated in Figure 5. With the aforementioned parameters of operations per second, thread and target operation parameters were sufficient to generate a burst that peaks the memory utilization at around 800 MB.

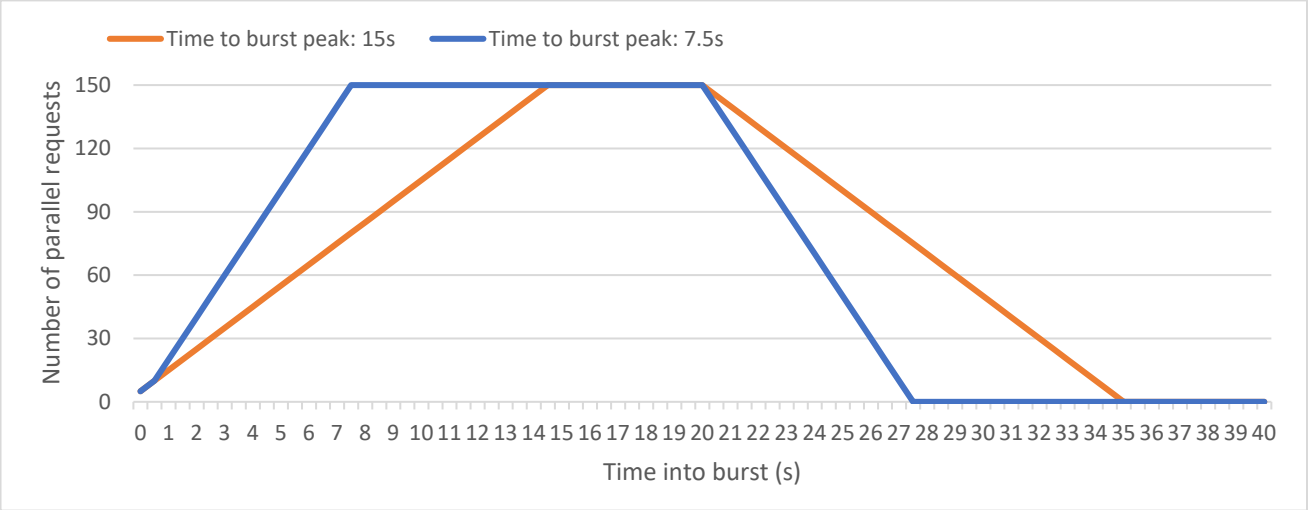


Figure 5. Intensity of generated burst workloads.

Once workloads start, the orchestrator monitors the cluster as described previously, initiating client connection transfer between appropriate nodes at appropriate memory utilization levels. Memory utilization on the pods is monitored. To simulate successful scaling, upon reaching 750 MB of memory utilization on either of the two operational nodes, the existing connections are gracefully transferred to the third node.

The client sessions are handled in three different ways:

1. **No action** taken due to the out-of-box functionality of the MySQL Galera Cluster and ProxySQL
2. **Redirecting** all new sessions to the second node
3. **Balancing** all sessions between two nodes

To summarize, we will evaluate our method with the parameters shown in Table 2.

Table 2. Parameters of the experiment.

Workload Type Distribution, %			Burst Handling Type	Time to Burst Peak (s)
Reads	**Updates**	**Inserts**		
83	15	2	No action	7.5
				15
			Redirect	7.5
				15
			Balance	7.5
				7.5

The YCSB collects quite an extensive list of statistics for each experiment run. We selected and aggregated the statistics necessary to measure availability: the number of total operations and the number of failed operations.

The total number of requests issued against the database has a consistent pattern; for example, the average for read requests is 85,976. However, there is a slight variation in each experiment; the total number of requests deviates from the average across all experiments by 2%. We did not consider this deviation to make an impact on the end results as we measured the failure rate percentage.

$$failure\ rate = \frac{number\ of\ failed\ requests}{total\ number\ of\ requests} * 100 \quad (1)$$

We ran the experiment five times with each parameter set to compare the error ratio and time under burst.

6. Results and Discussion

The first thing we noticed when analyzing the results of the experiment is that, despite the overall pattern, the deviation is quite significant. Thus, median values are important as well. The results show a non-negligible deviation, although, on average, there is an improvement in handling a burst condition if the proposed method is used. Even though the workload is synthetic and quite simple, without complex calculations, the memory utilization pattern lacks consistency. We were unable to control the inner workings of the XtraDB engine, which is a replacement for MySQL InnoDB. Therefore, in some of the experimental runs, the proposed method did not perform as expected. On the other hand, in some experimental runs, the cluster crashed almost immediately if no action was taken.

As expected, the proposed method of write-scaling for burst protection had an impact on availability in terms of successful requests and time compared to using just a single node for write requests. To begin with, the mean failure rate percentage, if no action to balance the workload was taken, ranged from 0.78 to 1.4 percent; if new sessions were redirected to another node, it ranged from 0.35 to 1.09 percent, depending on burst intensity (Figure 6). However, the proposed method allowed, for certain burst intensities, a reduction in the failure rate percentage to zero, and the mean rate reached a mere 0.04 percent.

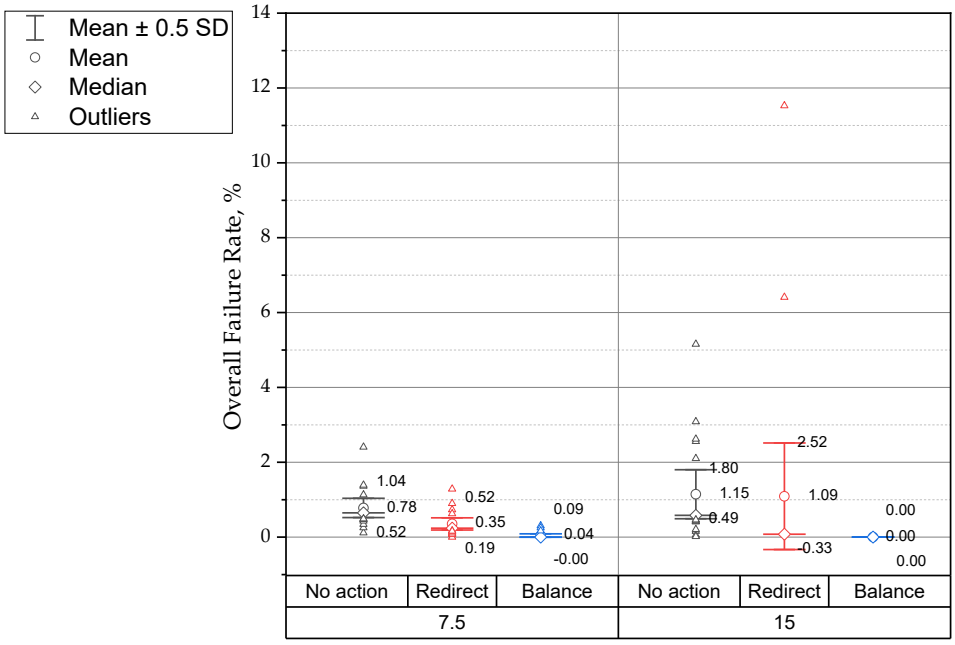


Figure 6. Dependency of overall failure rate on burst handling type and burst intensity.

For read requests (Figure 7), the proposed method reduced the impact of burst on the failure rate significantly. If no action was taken or only the burst workload was redirected,

the mean failure rate was between 0.33 and 0.61 percent. If sessions were balanced, the failure rate dropped to 0.04 percent.

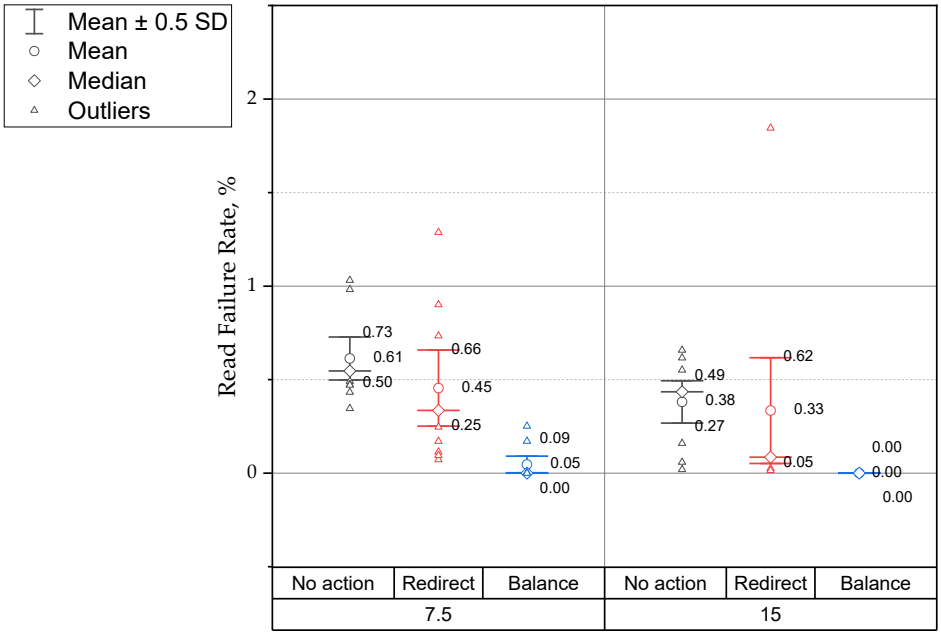


Figure 7. Dependency of read failure rate on burst handling type and burst intensity.

As with read requests, the proposed method allowed for a reduction in the impact of burst on the failure rate of updates as well (Figure 8). If no action was taken or only the burst workload was redirected, the mean failure rate would be between 0.17 and 2.36 percent. The highest update of the mean failure rate was for redirected update requests under lower intensity. However, the standard deviation was quite significant due to only one outlier. If sessions were balanced, the failure rate would be reduced to 0.01 percent.

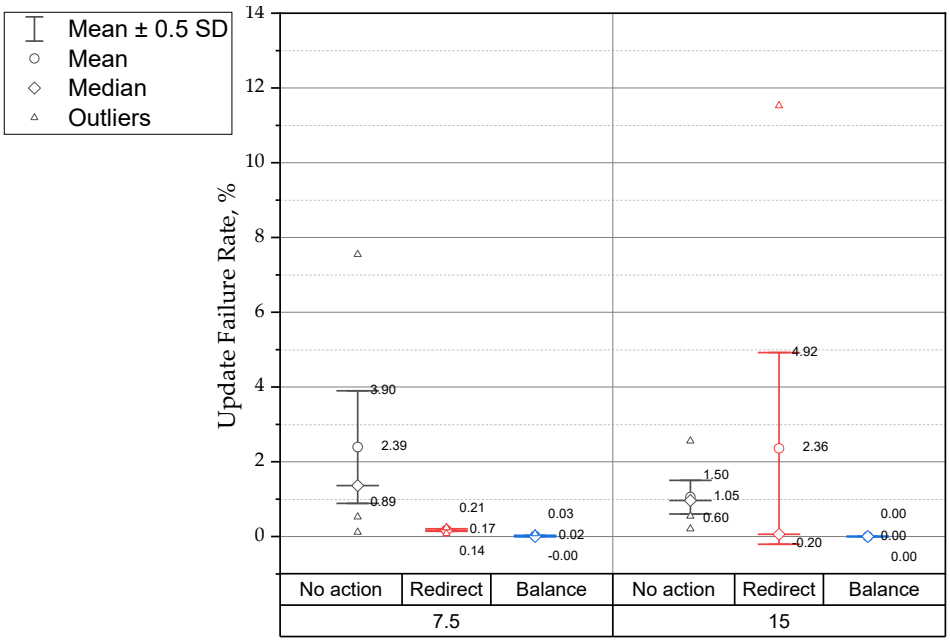


Figure 8. Dependency of update failure rate on burst handling type and burst intensity.

The failure rate of requests for data insertion (Figure 9) was reduced by the proposed method. Compared to a range of 0.33 to 2.77 percent of failed requests, the balanced

sessions allowed the failure rate to reduce to 0.61 percent. A couple of outliers skewed the results as well, but the trend remained the same.

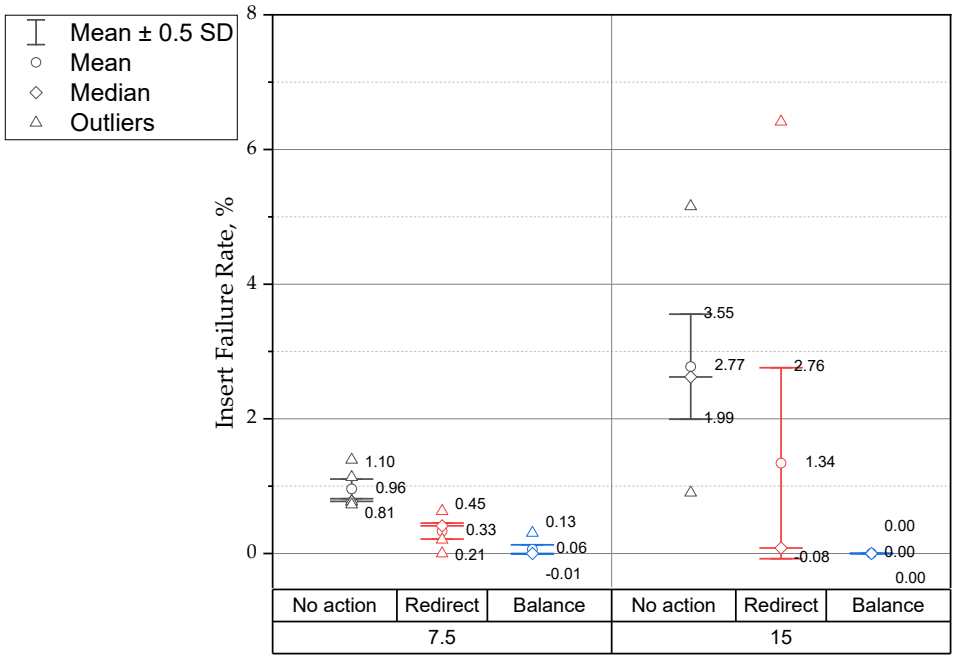


Figure 9. Dependency of insert failure rate on burst handling type and burst intensity.

To summarize, session balancing significantly reduces the amount of failure compared to taking no action or redirecting client sessions. As seen in Table 3, the failure rate decreased by at least 81.82%. Presumably, the lower load on a database node allowed the number of failed requests to be minimized when performing a graceful connection transfer to other nodes in the cluster. This finding confirms the results of our earlier research.

Table 3. Summary of failure rate.

Time to Burst Peak (s)	Workload	Average Failure Rate by Burst Handling Type			Comparative Decrease in Failure Rate Using Balancing (%)	
		No Action	Redirect	Balance	No Action	Redirect
7.5	Reads	0.61	0.45	0.05	91.80	88.89
	Updates	2.39	0.17	0.02	99.16	88.24
	Inserts	0.96	0.33	0.06	93.75	81.82
	All	0.78	0.35	0.04	94.87	88.57
15	Reads	0.38	0.33	0	100	100
	Updates	1.05	2.36	0	100	100
	Inserts	2.77	1.34	0	100	100
	All	1.15	1.09	0	100	100

The time needed for any of the nodes to reach 750 MB in memory utilization was improved alongside the proposed method (Figure 10). The time under burst increased to 42.4 and 40.8 s under burst conditions compared to 23 and 25 s if no action was taken. Redirecting burst sessions only also resulted in an increased time under burst compared to taking no action. However, the increase in time under burst at higher load intensities when using balancing compared to redirecting was insignificant. While redirected sessions allowed the cluster to operate under burst for 42.2 s on average, when sessions were balanced, the average time under burst was 40.8 s. Judging by the outliers, the time under burst may vary for each of the approaches.

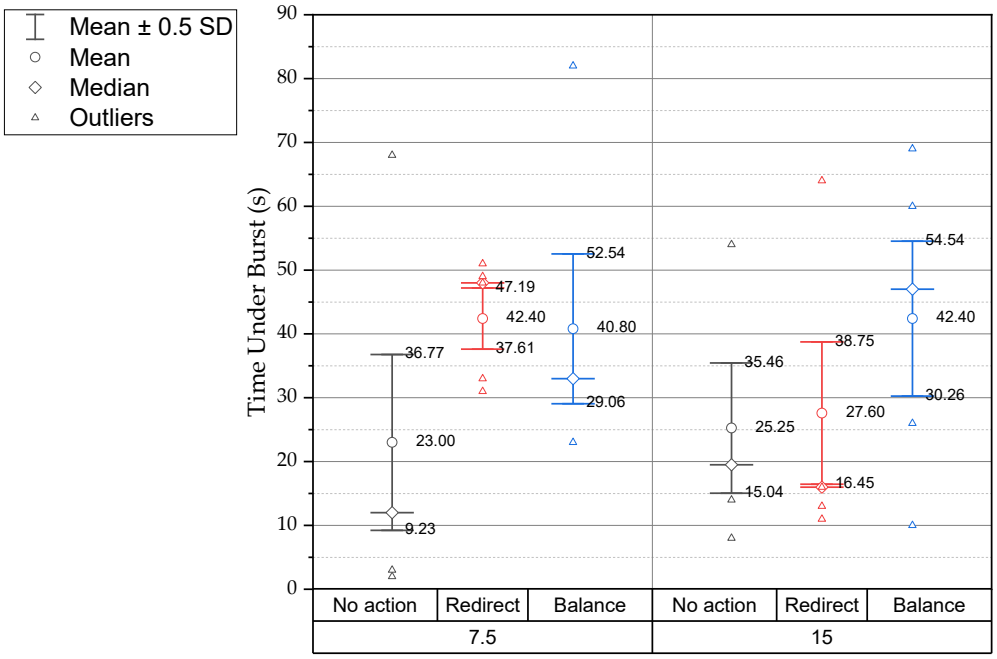


Figure 10. Dependency of time under burst-on-burst handling types and burst intensity.

Despite the outliers, there is still a positive trend for time under burst changes when balancing is used (Table 4). The combination of lower failure rate percentages and increased or slightly diminished times under burst, depending on the methods compared, showed that session balancing is a viable approach to increase stateful service availability under burst conditions.

Table 4. Summary of time under burst.

Time to Burst Peak (s)	Average Time Under Burst			Comparative Increase in Time Under Burst when Using Balancing (%)	
	No Action	Redirect	Balance	No Action	Redirect
7.5	23	42.2	40.8	177.39	96.68
15	25.25	27.6	42.4	167.92	153.62

One thing in common between all types of workloads is the number of outliers. In some cases, the outliers show a significant deviation from mean values and the overall trend, which, despite the outliers, is persistent across the experiments. As MySQL memory usage depends on many factors, in certain cases, it may vary from the baseline. For example, if a new set of data or query plans are added to the cache. The disadvantages of the proposed method stem from the use of a Multi-Primary relational database management system. It exhibits diminished performance due to additional complexity and the increased communications latency compared to Single-Primary replication configurations. Unlike the Single-Primary replication setup, latency is further increased by burst workloads. Conflict resolution adds an additional degree of latency in a Multi-Primary replication setup when under peak loads. This is partially mitigated by setting up one node designated as Pseudo-Primary. This approach would reduce the necessity for conflict resolution, as only a single database node would be responsible for handling the write operations. As seen in Figure 11, average latency is increased when workloads are balanced between two nodes. The additional latency is introduced due to the increased volume of bi-directional replication as multiple nodes become processing writes and also at the end when client connections are transferred to the third node. Even though the latency is higher compared to when no

action is taken, it only increases when client sessions are balanced; thus, the SLOs related to a stateful microservice latency are degraded for a limited duration of time.

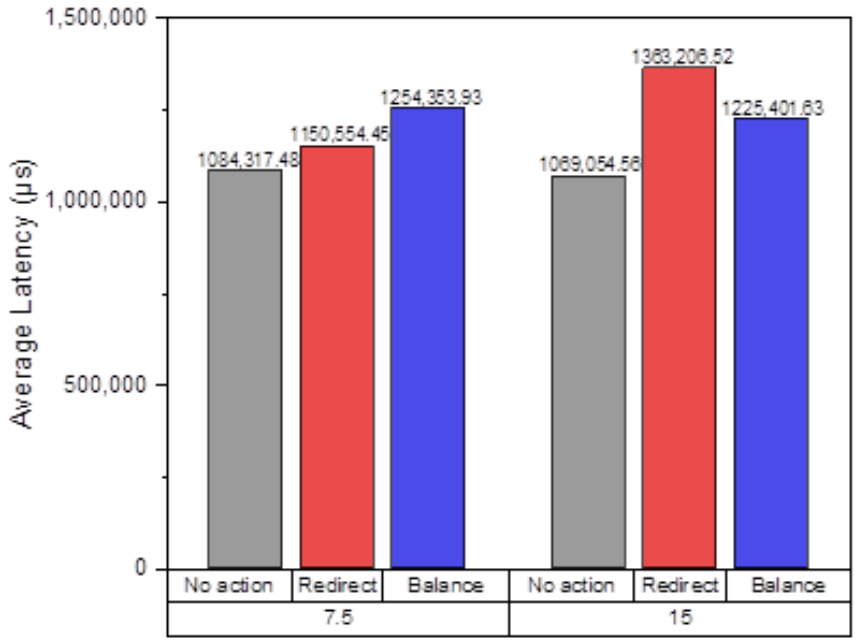


Figure 11. Dependency of average latency (µs) on burst handling type and burst intensity.

However, in terms of handling a burst, the proposed method using the Multi-Primary database offers several benefits. To begin with, a stateful microservice operator can handle a burst independently with no changes needed on the client side. It is a predictable as well as transparent rule-based approach, and the reasons for these actions taken are clear and understandable. Although to be unbiased, there is still uncertainty as to how MySQL, or any other RDBMS for that matter, would operate and use memory under peak workload conditions. Finally, the proposed method was proven to work using open-source components, and there were other software combinations that were likely to function in the same or an even better manner as an implementation of the proposed method.

There are a couple of avenues for further research. To begin with, there are a number of other database management systems that support Multi-Primary replication, for example, PostgreSQL with Bi-Directional Replication extension, CockroachDB, CouchDB, and Cassandra. PostgreSQL is one of the "classic" RDBMSs; thus, it is likely that the proposed method could achieve results similar to the MySQL Galera Cluster. CockroachDB is a more modern RDBMS, built as a distributed database with a modern consensus protocol and automatic sharding. Although a more modern tool, CockroachDB still relies on third-party proxy solutions to balance the workload, which allows the proposed method to be employed to handle bursts. Cassandra and CouchDB, two NoSQL alternatives enabling Multi-Primary replications, are built with data distribution in mind. However, promoting a Cassandra node to a writer requires additional reconfiguration, while the CouchDB node must be configured in advance to accept writes. This would likely cause difficulties with Cassandra and CouchDB in case the proposed method is to be used to handle bursts. An inquiry could be made into the efficiency of the proposed method using a database management system other than MySQL Galera Cluster.

Additionally, the integration of the proposed rule-based method with ML-based techniques can enhance the way burst workloads are handled while optimizing resource usage. It may act as a failsafe measure given that workload prediction may be imprecise and a stateful microservice is not properly adapted to the actual demand. To begin with,

the proposed rule-based method can extend the operational time of a stateful microservice if vertical scaling is not initiated on time. The extra time under burst would, hopefully, be sufficient for the stateful microservice to scale. Secondly, the balancing of the workload may be initiated while the microservice scales vertically.

7. Conclusions

The proposed method of using write-scaling and load balancing to protect a stateful microservice under an overwhelming burst workload allowed us to increase its availability. The additional time under burst, although modest, in certain cases could be sufficient to allow vertical scaling to complete and thus would enable the stateful microservice to handle the extra workload.

Our experiments have shown that stateful microservices can operate under burst for a longer duration. Burst intensity does make an impact on availability. If the burst intensity is low enough, the load can be balanced and writes can be scaled to the point where the stateful microservice scales in time to match the increased demand. On the other hand, if the burst is sufficiently intensive, a stateful microservice crashes regardless of whether or not the proposed method is used. However, the failure rate percentage is still lower, and the service is unavailable for shorter amounts of time. The proposed method allowed a stateful microservice to operate under a burst for almost twice as long compared to standard functionality. Although there is a response latency increase if the proposed method is used, the stateful microservice is affected by it only for a limited amount of time.

In environments where resources are constrained, a stateful microservice can be sized to process the presumed standard workload and scaled up to meet the increased demand. In addition, the proposed method can be used as a failsafe if employed workload prediction algorithms are imprecise in workload forecast.

In this paper, we have shown that stateful microservices can operate under a sudden and significant workload, which would otherwise potentially cause the system to crash for longer durations.

Author Contributions: Methodology, D.M.; Software, K.P.; Investigation, K.P.; Writing—original draft, K.P.; Writing—review and editing, D.M.; Supervision, D.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Data are contained within the article.

Acknowledgments: The authors would like to express their sincere gratitude to Jonathan Stephen Salter for his invaluable assistance and insightful suggestions that greatly improved the quality of this manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Manouvrier, M.; Pautasso, C.; Rukoz, M. *Microservice Disaster Crash Recovery: A Weak Global Referential Integrity Management*; Springer International Publishing: Cham, Switzerland, 2020; Volume 12138. [CrossRef]
2. Truyen, E.; Van Landuyt, D.; Lagaisse, B.; Joosen, W. Performance Overhead of Container Orchestration Frameworks for Management of Multi-Tenant Database Deployments. In Proceedings of the 34th ACM Symposium on Applied Computing, Limassol, Cyprus, 8–12 April 2019; pp. 156–159.
3. Pardon, G.; Pautasso, C.; Zimmermann, O. Consistent Disaster Recovery for Microservices: The BAC Theorem. *IEEE Cloud Comput.* **2018**, *5*, 49–59. [CrossRef]
4. Pereira, P.A.; Bruce, M. *Microservices in Action*; Manning: Shelter Island, NY, USA, 2019.
5. Newman, S. *Building Microservices: Designing Fine-Grained Systems*, 1st ed.; O'Reilly Media: Sebastopol, CA, USA, 2015.

6. Kubernetes. Run a Replicated Stateful Application. *Kubernetes.io*. Available online: https://kubernetes.io/docs/tasks/run-application/run-replicated-stateful-application/ (accessed on 6 March 2021).
7. Kannaiah, G.M. *Kubernetes Anti-Patterns: Overcome Common Pitfalls to Achieve Optimal Deployments and a Flawless Kubernetes Ecosystem*, 1st ed.; Packt Publishing: Birmingham, UK, 2024.
8. Shen, H.; Chen, L. Resource Demand Misalignment: An Important Factor to Consider for Reducing Resource Over-Provisioning in Cloud Datacenters. *IEEE/ACM Trans. Netw.* **2018**, *26*, 1207–1221. [CrossRef]
9. Rubak, A.; Taheri, J. Machine Learning for Predictive Resource Scaling of Microservices on Kubernetes Platforms. In Proceedings of the 16th IEEE/ACM International Conference on Utility and Cloud Computing, UCC 2023, Orlando, FL, USA, 4–7 December 2023; Association for Computing Machinery, Inc.: New York, NY, USA, 2023. [CrossRef]
10. Dang-Quang, N.M.; Yoo, M. Deep learning-based autoscaling using bidirectional long short-term memory for Kubernetes. *Appl. Sci.* **2021**, *11*, 3835. [CrossRef]
11. Lassnig, M.; Fahringer, T.; Garonne, V.; Molfetas, A.; Branco, M. Identification, modelling and prediction of non-periodic bursts in workloads. In Proceedings of the 10th IEEE/ACM International Conference on Cluster, Cloud, and Grid Computing, Melbourne, Australia, 17–20 May 2010; pp. 485–494. [CrossRef]
12. Deshpande, U. Caravel: Burst Tolerant Scheduling for Containerized Stateful Applications. In Proceedings of the 39th IEEE International Conference on Distributed Computing Systems, Dallas, TX, USA, 1–4 July 2019; pp. 1432–1442. [CrossRef]
13. Campbell, L.; Majors, C. *Database Reliability Engineering*, 1st ed.; O'Reilly Media: Sebastopol, CA, USA, 2017.
14. Pakrijauskas, K.; Mažeika, D. A Method of Transparent Graceful Failover in Low Latency Stateful Microservices. *Electronics* **2022**, *11*, 3936. [CrossRef]
15. Ali-Eldin, A.; Seleznjev, O.; Sjostedt-de Luna, S.; Tordsson, J.; Elmroth, E. Measuring Cloud Workload Burstiness. In Proceedings of the 7th IEEE/ACM International Conference on Utility and Cloud Computing, London, UK, 8–11 December 2014; IEEE: Piscataway, NJ, USA, 2024; pp. 566–572. [CrossRef]
16. Bodik, P.; Patterson, D.A.; Jordan, M.I.; Franklin, M.J.; Fox, A. Characterizing, modeling, and generating workload spikes for stateful services. In Proceedings of the 1st ACM Symposium on Cloud Computing, Indianapolis, IN, USA, 10–11 June 2010; ACM Digital Library: New York, NY, USA, 2010; pp. 241–252.
17. Woodruff, J.; Moore, A.W.; Zilberman, N. Measuring Burstiness in Data Center Applications. In Proceedings of the 2019 Workshop on Buffer Sizing, New York, NY, USA, 9 December 2019; ACM: New York, NY, USA, 2019; pp. 1–6. [CrossRef]
18. Baarzi, A.F.; Kesidis, G. SHOWAR: Right-sizing and efficient scheduling of microservices. In Proceedings of the 2021 ACM Symposium on Cloud Computing (SoCC 2021), Seattle, WA, USA, 1–4 November 2021; ACM: New York, NY, USA, 2021; pp. 427–441. [CrossRef]
19. Iqbal, W.; Erradi, A.; Mahmood, A. Dynamic workload patterns prediction for proactive auto-scaling of web applications. *J. Netw. Comput. Appl.* **2018**, *124*, 94–107. [CrossRef]
20. Xie, S.; Wang, J.; Li, B.; Zhang, Z.; Li, D.; Hung, P.C.K. PBScaler: A Bottleneck-Aware Autoscaling Framework for Microservice-Based Applications. *IEEE Trans. Serv. Comput.* **2024**, *17*, 604–616. [CrossRef]
21. Kubernetes. Assign Memory Resources to Containers and Pods. Available online: https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource/ (accessed on 7 May 2025).
22. Oracle. How MySQL Uses Memory. *MySQL.com*. Available online: https://dev.mysql.com/doc/refman/8.0/en/memory-use.html (accessed on 7 May 2025).
23. Percona LLC. *Percona XtraDB Cluster Documentation*; Percona LLC.: Durham, NC, USA, 2020.
24. Codership Ltd. Replication API. Available online: https://galeracluster.com/library/documentation/architecture.html#wsrep-api (accessed on 7 May 2025).
25. Wang, Z. Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices. Available online: https://www.usenix.org/conference/nsdi24/presentation/wang-zibo (accessed on 12 May 2025).
26. GitHub. Burst Tolerance for Stateful Microservices. Available online: https://github.com/kespak/burst-tolerance-for-stateful-microservices (accessed on 1 July 2025).
27. Hauer, T.; Hoffman, P.; Lunney, J.; Ardelean, D.; Diwan, A. Meaningful Availability. In Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), Santa Clara, CA, USA, 25–27 February 2020; pp. 545–557. Available online: https://www.usenix.org/conference/nsdi20/presentation/hauer (accessed on 24 May 2025).
28. Cooper, B.F.; Silberstein, A.; Tam, E.; Ramakrishnan, R.; Sears, R. Benchmarking cloud serving systems with YCSB. In Proceedings of the 1st ACM Symposium on Cloud Computing, in SoCC '10, Indianapolis, IN, USA, 10–11 June 2010; Association for Computing Machinery: New York, NY, USA, 2010; pp. 143–154. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.